

Code Documentation

Group Number - 22

Github Link: https://github.com/AgrawalDrishti/SDE_Mini_Project

In our project, Deployment Bot we have created three services, namely **Upload service**, **Build service** and **Host service**.

1. **Upload Service:** To upload the project cloned from github url, zip folders to the Google Cloud Platform (GCP) buckets. For this, we created three endpoints :

A. **github_url_handler**

It validates the URL, clones the repository to a local directory, compresses it into a zip file, and uploads it to a Google Cloud Bucket.

Parameters

- req (Object): The request object containing:
body.github_url (String): The URL of the GitHub repository to process.
- res (Object): The response object used to send back status and result messages.

Process Flow

1. Validation:

- a. Checks if the request body is empty.
- b. Validates the github url format using the helper function 'is_valid_github_url_repo'.

2. Github Repository Check :

Uses '*axios*' to send a GET request to the provided GitHub URL to ensure it is accessible.

3. Repository Cloning and Zipping :

- a. Creates a unique folder name using a SHA-256 hash of the URL.
- b. Clones the repository to a local directory using '*simpleGit*'.
- c. Compresses the cloned directory into a zip file.

4. Upload and Queue:

- a. Uploads the zip file to a Google Cloud Bucket with a specified destination path.
- b. Sends a POST request to an external queue service to enqueue the zipped repository for further processing.

5. Cleanup :

Removes the cloned repository folder and zip file from the local system.

Returns

- a. **200 OK**: If the repository is successfully cloned, zipped, uploaded, and queued.
- b. **400 Bad Request**: If the GitHub URL is missing, invalid, or if cloning, zipping, or uploading fails.
- c. **404 Not Found**: If the specified GitHub repository URL does not exist.
- d. **500 Internal Server Error**: For unexpected server errors.

B. zip_file_handler

It validates the file, checks its contents for unwanted directories (e.g., node_modules), uploads it to a Google Cloud Bucket, and then adds it to a queue for further processing.

Parameters

- req (Object): The request object containing:
files.codebase (**File**): The zip file uploaded by the user.
- res (Object): The response object used to send back status and result messages.

Process Flow

1. Validation:

- a. Checks if the **files** property in the request is null, empty, or does not contain a file named **codebase**.

2. File Information :

Logs the file name, size, and MIME type.

3. MIME type check:

Checks if the uploaded file's MIME type is one of the acceptable zip file types (**application/zip**, **application/x-zip-compressed**, **application/x-zip**, **application/zip-compressed**, **application/octet-stream**).

4. File Upload and Queue:

- a. Generates a unique destination path for the zip file using a SHA-256 hash of the file name.
- b. Saves the zip file to a local path.
- c. Uploads the zip file to a Google Cloud Bucket.
- d. Sends a POST request to an external queue service to enqueue the zip file for further processing.

5. CleanUp:

Removes the local copy of the zip file after uploading.

Returns

- a. **200 OK**: If the file is successfully validated, uploaded, and queued.

- b. **400 Bad Request**: If the uploaded file is missing, has an incorrect format, is not a zip file, or contains a node_modules directory.
- c. **500 Internal Server Error**: For unexpected server errors.

C. **docker_image_handler**

It checks if the provided Docker image exists on DockerHub, and if valid, it sends a success response.

Parameters

- req {Object}: request object, containing the Docker image details in the body.
- res {Object}: response object used to send the result of the operation.

Process Flow

1. Validation:

- a. Checks if the request body is empty.
- b. Uses **check_invalid_docker_image** to validate the **docker_image** field.

2. Docker Hub Check:

- a. Constructs the Docker Hub URL using the provided **docker_image** name.
- b. Uses **'axios'** to send a GET request to the Docker Hub URL to check if the image exists.

Returns

- a. **200 OK**: If the Docker image is successfully validated and found on Docker Hub.
- b. **400 Bad Request**: If the request body is empty or if the Docker image validation fails.
- c. **404 Not Found**: If the specified Docker image is not found on Docker Hub.
- d. **500 Internal Server Error**: For unexpected server errors.

2. **Build Service**: To containerize the project and build its docker image, we have created this service. For this, we will fetch the zip folders from the GCP bucket using the endpoint :

A. **get_zip_url**

An asynchronous API endpoint function that communicates with a queue service to retrieve (dequeue) the next zip file URL to be processed.

Parameters

- req (Object): The request object
- res (Object): The response object used to send back status and result messages.

Process Flow

1. **Queue Service Request:**

- a. Uses '*axios*' to send a GET request to the queue service endpoint (*/dequeue*) to retrieve the next zip file URL.

2. **Handling Success:**

If the GET request is successful, the function sends a response containing the message retrieved from the queue service.

Returns

- a. **200 OK:** If the request to the queue service is successful.
- b. **Error:** If the request to the queue service fails, the function returns an error object with details about the failure.

The build service uses some helper functions like :

1. **downloadAndUnzipFile(destinationFile):**

Downloads a zip file from a Google Cloud Storage bucket using the provided destination path, extracts the contents of the zip file, and prepares the files for further processing.

2. **build(project_zip_url):**

Handles the entire build process:

- a. Calls *downloadAndUnzipFile()* to download and extract the project.
- b. Deletes the *.git* folder to clean up the project directory.
- c. Generates a *Dockerfile* for containerizing the project.
- d. Logs into Docker, builds a Docker image, tags it, and pushes the image to Docker Hub.

3. **startBuilding() :**

Continuously checks the queue for new projects to build. It uses a *mutex* for thread-safe access, dequeues the next project to build, and then calls the *build()* function. If no project is available, it sleeps for 5 seconds before checking again.

3. **Host Service:** Once the docker image is pushed to docker hub, the host service will start hosting on the google cloud run using the below function :

A. **deploy(img) :**

Deploys a Docker image to Google Cloud Run using the *gcloud* command. This function runs the deployment command asynchronously in a child process and logs any errors or outputs.

Parameters

- *img* (String): The name of the Docker image to deploy

Process Flow

- a. Executes the `gcloud run deploy` command using the specified Docker image.
- b. Deploys the application as a Google Cloud Run service in the `asia-south2` region.
- c. Configures the service to be managed, accessible without authentication, and listens on port `3000`.

B. `startDeploying()`:

Continuously checks the hosting queue for new Docker images to deploy. If an image is available, it calls the `deploy()` function to deploy the image to Google Cloud Run. Uses a mutex to ensure only one deployment process runs at a time.

Process Flow

- a. Runs an infinite loop to constantly check for projects to deploy.
- b. **Mutex:** Uses a mutex to ensure that only one deployment process can run at a time.
- c. **Queue Check:**
 1. Call the service (`/dequeue`) to get the next image to deploy from the hosting queue.
 2. If no project is available (`message` is `null`), it logs a message and sleeps for 5 seconds before retrying.
- d. **Deploy:** If a project is found in the queue, logs the message and calls the `deploy()` function with the Docker image URL.
- e. **Release Mutex:** Releases the mutex after each iteration to allow other processes to access the deployment queue.